Biais de surestimation dans DDPG et TD3 sur LunarLander : la layer normalization aide-t-elle ?

Thomas Anato

1 Les deux algorithmes

DDPG (Lillicrap et al., 2016) est du Q-learning pour des actions continues. En Q-learning la politique se lit dans le critique, $\pi(s) = \arg\max_a Q(s,a)$, et la cible s'écrit $y = r + \gamma \max_{a'} Q(s',a')$. Ce maximum est gratuit quand les actions sont en nombre fini, mais devient un problème d'optimisation à chaque pas quand elles sont continues. DDPG le remplace par un second réseau, l'acteur μ_θ, entraîné à monter le gradient de $Q_\phi(s, \mu_\theta(s))$. L'acteur est donc un *argmax appris*.

TD3 (Fujimoto et al., 2018) y ajoute trois correctifs destinés à combattre la surestimation du critique : deux critiques Q_1 et Q_2 dont on retient le minimum dans la cible, une mise à jour de l'acteur une fois sur d seulement, et un bruit ajouté à l'action dans le calcul de la cible.

L'idée de tester la layer normalization (Ba et al., 2016) vient de Nauman et al. (2024) : sur une soixantaine de variantes régularisées de SAC, une LN dans le critique réduit la surestimation *davantage* que les techniques conçues pour ça. Ce projet demande si cela vaut aussi pour DDPG et TD3.

2 D'où vient le biais

Une fois entrainé, l'erreur du critique est $\varepsilon(s,a) = Q_\phi(s,a) - Q^\pi(s,a)$. Au départ, cette erreur fluctue de manière aléatoire, sans pencher particulièrement vers la hausse ou la baisse, mais ce n'est plus le cas quand on introduit une opération de maximisation. Si l'on choisissait les actions au hasard, ces erreurs finiraient par s'annuler. Mais comme l'acteur cherche activement à maximiser les récompenses, il se dirige naturellement vers les actions que le critique surévalue le plus. Donc pour deux valeurs réelles équivalentes, il sélectionne les points où l'erreur ε est la plus forte. **Il sélectionne les plus favorables** : $\mathbb{E}[\max_a \hat{Q}] \geq \max_a \mathbb{E}[\hat{Q}]$, avec inégalité stricte dès que les estimations sont bruitées. L'absence d'opérateur max explicite dans DDPG ne supprime donc pas le biais. À ce phénomène s'ajoute le problème de l'extrapolation. Le réseau critique n'a aucune contrainte sur la valeur qu'il attribue aux actions qu'il n'a encore jamais explorées. Or, ce sont ces actions aux prédictions incertaines et potentiellement surestimées que l'acteur risque d'exploiter à tort.

Le bootstrapping propage ensuite l'erreur, et de façon asymétrique. En effet, sous-estimation n'est pas transmise puisque l'acteur évite les régions où le critique est bas et que ces valeurs n'entrent dans aucune cible. L'actualisation fournit un rappel, puisque pour un biais uniforme δ la cible vaut $Q^*(s) + \gamma\delta$. En notant ϵ le biais réinjecté à chaque mise à jour, le point fixe est

$$\delta_{\text{eq}} = \epsilon + \gamma\, \delta_{\text{eq}} \quad \Longrightarrow \quad \delta_{\text{eq}} = \frac{\epsilon}{1-\gamma}, \tag{1}$$

soit une amplification d'un facteur 50 à $\gamma = 0{,}98$. On s'attend donc à un plateau, pas à une divergence.

Un dernier point: l'acteur ne dépend que des *variations* de Q_ϕ en a, pas de son niveau. Un biais uniforme ne change donc pas la politique et seul un biais *inégal*, qui crée des maxima artificiels, la dégrade. Rien ne garantit a priori que l'amplitude du biais prédise la performance, et c'est pourquoi nous suivrons aussi son écart-type. C'est l'argument de Chen et al. (2021), pour qui l'uniformité du biais compte autant que son niveau.

3 Comment nous mesurons le biais

Le critique prédit le retour actualisé. Cette prédiction est vérifiable, pour cela il suffit de dérouler la trajectoire et de sommer les récompenses réellement reçues. À chaque point de mesure, nous déroulons donc 10 épisodes déterministes, sans bruit d'exploration, et nous comparons

$$\text{biais}(s_t, a_t) = Q_\phi(s_t, a_t) - G_t, \qquad G_t = \sum_{k \geq 0} \gamma^k r_{t+k+1}. \tag{2}$$

Chaque point de mesure fournit ainsi un nuage de 1 000 à 5 000 couples état-action, dont nous retenons la moyenne et l'écart-type.

Deux précisions: G_t n'est pas la valeur exacte $Q^\pi(s_t, a_t)$ puisque les transitions sont stochastiques et le terrain est retiré à chaque réinitialisation sans être observé, donc G_t en est un échantillon non biaisé mais bruité. Pour TD3, Q_ϕ désigne le *minimum ponctuel* des deux critiques, la quantité qui entre réellement dans la cible.

La troncature LunarLander coupe les épisodes à $T_{\max} = 1000$ pas. Cette limite de temps n'est pas un état terminal, et le critique la traverse en bootstrappant (Pardo et al., 2018) alors que le G_t des instants tardifs, lui, est incomplet. Nous ne gardons que les instants $t \leq T_{\max} - H$ avec $H = 500$ ($\gamma^{500} \approx 4 \cdot 10^{-5}$, queue négligeable). Ce critère ne dépend que de t, jamais de la façon dont l'épisode s'est terminé.

Pas de normalisation. Dans beaucoup de papier, on utilise le biais relatif $\overline{Q-G}/|\bar{G}|$, inutilisable ici car $\bar{G}$ traverse zéro. Cela produit donc un biais arbitrairement grand sans que cela n'implique un phénomène particulier. Nous utiliserons donc le biais brut.

Vérifications. Avant d'analyser les résultats, nous avons effectué quelques contrôles sur notre code. D'abord, le calcul des retours actualisés a été validé sur un petit exemple à la main, puis en rejouant un épisode complet action par action. Nous nous sommes aussi assurés que notre règle pour couper les fins d'épisodes dépendait uniquement de t. Par ailleurs, à l'initialisation (avant le tout premier pas d'apprentissage), le critique prédit logiquement des valeurs presque nulles ($\max|\overline{Q}| = 0,80$) face à des retours vrais $\overline{G}$d'environ 75. Le biais calculé donne donc logiquement $-\overline{G}$, ce qui confirme

que la formule fonctionne. Enfin, pour vérifier notre utilisation de *Stable-Baselines3*, nous avons paramétré TD3 sans ses correctifs (un seul critique, d=1, pas de bruit de cible) et constaté qu'il donnait exactement les mêmes résultats que DDPG.

Statistiques et reproductibilité. Chaque configuration est évaluée sur plusieurs graines d'initialisation, pour que nos observations ne dépendent pas des conditions de départ d'un réseau particulier. Nos figures montrent donc la médiane entourée de la bande interquartile plutôt qu'une moyenne, et nos tableaux la moyenne interquartile (IQM) avec un intervalle bootstrap à 95 %, deux résumés peu sensibles aux exécutions extrêmes. La *Layer Normalization* (LN) ne consommant aucun tirage aléatoire supplémentaire, les réseaux avec et sans LN partent des mêmes poids pour une même graine : les comparaisons sont *appariées*, et nous donnons le nombre de graines concordantes (test des signes) à côté d'un test de Mann–Whitney non apparié, qui compare les deux groupes par le classement de tous leurs scores. Ces choix suivent les recommandations de Colas et al. (2019). Nos petits échantillons imposent des planchers : un résultat unanime survient par pur hasard avec probabilité $1/2^5 = 3{,}1\,\%$ sur cinq graines et $1/2^4 = 6{,}25\,\%$ sur quatre, et le Mann–Whitney ne descend pas sous $1/126 = 0{,}008$ en 4 contre 5, ni sous $1/252 = 0{,}004$ en 5 contre 5. Aucune valeur p de ce rapport ne peut leur être inférieure. Enfin, notre démarche est exploratoire : chaque expérience a été conçue pour analyser les résultats de la précédente.

4 Expérience 1 : les quatre configurations

Les hyperparamètres, et pourquoi ceux-là. Nous partons des valeurs du *RL Baselines3 Zoo*, le dépôt d'hyperparamètres compagnon de Stable-Baselines3, qui les a réglées pour LunarLanderContinuous. Elles nous évitent de régler nous-mêmes un algorithme dont nous voulons étudier le comportement, et le Zoo donne *les mêmes* valeurs pour DDPG et TD3, donc aucun n'est avantagé par un réglage propre. Seule l'architecture change, $[400, 300]$ dans le Zoo, ramenée à $[256, 256]$ pour le budget de calcul. On remarquera $\gamma = 0{,}98$, qui d'après l'équation (1) amplifie d'un facteur 50 tout biais réinjecté.

La layer normalization est insérée dans le critique seul, sous la forme `Linear` → `LayerNorm` → `ReLU` sur toutes les couches sauf la sortie. Quatre configurations, 5 graines, 200 000 pas (la graine 4 de DDPG sans LN n'a pas abouti, d'où $n = 4$ dans cette seule condition).

Table 1: Hyperparamètres. Identiques pour les quatre configurations, sauf les trois dernières lignes, propres à TD3.

Environnement	`LunarLanderContinuous-v3`	Architecture, lot	$[256, 256]$ ReLU, 256
Pas d'env. ; mesure tous les	200 000 ; 10 000 (10 épisodes)	Optimiseur ; τ	Adam 10^{-3} ; 0,005
γ ; horizon H	0,98 ; 500	Nombre de critiques	2 (TD3) / 1 (DDPG)
Buffer ; `learning_starts`	200 000 ; 10 000	Retard de l'acteur d	2
Pas de gradient / pas d'env. ; bruit	1 ; gaussien $\sigma = 0{,}1$	Bruit de cible	0,2, écrêté à 0,5

Résultat. Il est de signe opposé selon l'algorithme (tableau 2, figure 1).

Sur DDPG, la LN *aggrave* la surestimation : le biais passe de 47,6 à 130,5, son ecar-type double, la performance chute de 162 points, et les quatre graines vont dans le même sens pour ces trois quantités (Mann–Whitney $p = 0{,}016$ sur le biais et sur le retour, $p = 0{,}008$ sur l'écart-type). Sur TD3, elle *ne change pas* le biais (+1,0, trois graines sur cinq seulement dans ce sens), et améliore pourtant le retour sur les **cinq** graines (+79,3, test des signes $p = 0{,}031$).

On peut faire deux constatations supplémentaire en plus de simplement observer le biais: TD3 *sous-estime* alors que DDPG surestime, et DDPG sans LN est la meilleure configuration à 200 000 pas, ce qui contredit le résultat habituel TD3 > DDPG. Nous y revenons en section 6 et 8 .

Table 2: Expérience 1. Valeurs finales (moyenne des points 180k/190k/200k), IQM [bootstrap 95 %]. En bas, l'effet apparié de la LN et le nombre de graines concordantes.

		Retour(performance)	$\overline{Q-G}$	sd$(Q-G)$	Troncature
DDPG	(n=4)	**158** [19, 234]	47,6 [29, 86]	35,0	0,03
DDPG + LN	(n=5)	−40 [−85, 52]	130,5 [93, 202]	63,2	0,02
TD3	(n=5)	45 [−28, 145]	−6,4 [−8,8, −5,0]	13,4	0,51
TD3 + LN	(n=5)	**155** [53, 194]	−5,7 [−7,7, −3,6]	14,0	0,22
effet de la LN sur DDPG		−162 (4/4)	+93,4 (4/4)	+32,5 (4/4)	0,00 (2/4)
effet de la LN sur TD3		+79,3 (5/5)	+1,0 (3/5)	+0,7 (3/5)	−0,23 (4/5)

Ce qu'on en conclut. L'effet de la LN sur le biais dépend de l'algorithme. Sur DDPG elle l'aggrave et fait chuter la performance tandis sur TD3 elle ne le touche pas et améliore pourtant le retour de toutes les graines, donc son bénéfice repose sur autre chose que le contrôle du biais. Cela répond à la première question mais pas à la seconde : les configurations les plus biaisées sont les moins performantes, sans que nous puissions dire que le biais en est la cause.

5 Expérience 2 : fabriquer du biais pour voir s'il nuit

Les configurations les plus biaisées sont aussi les moins performantes, mais cela ne prouve rien puisque les deux pourraient avoir une cause commune. Pour trancher il faut *injecter* du biais sans rien changer d'autre. Pour cela nous allons faire varier le ratio UTD (Update-To-Data), c'est-à-dire le nombre de mises à jour du réseau effectuées pour chaque nouvelle interaction avec l'environnement. En effet, plus de mises à jour par transition collectée augmente le terme ϵ de l'équation (1) sans toucher à l'architecture, à l'objectif ni aux données. Nous reprenons DDPG sans LN sur 100 000 pas

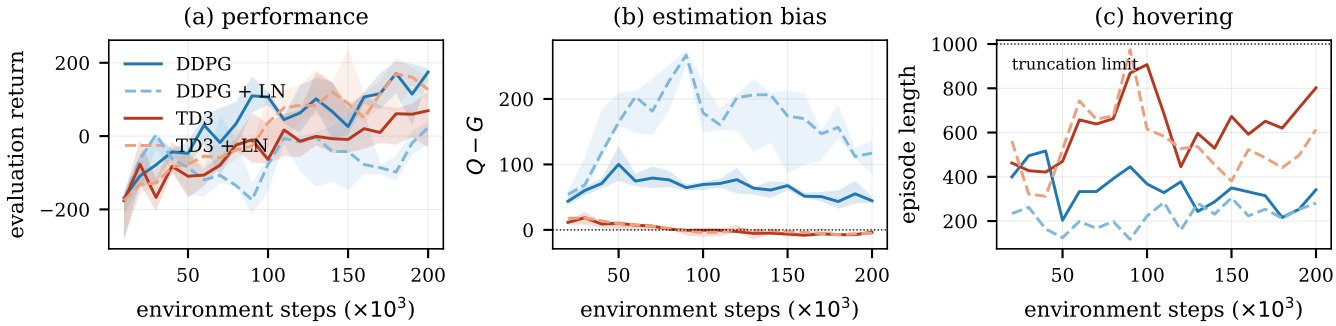


Figure 1: Expérience 1. Médiane sur les graines, bande interquartile. (a) performance ; (b) biais ; (c) longueur d'épisode, la ligne pointillée marquant la troncature. La LN monte DDPG en (b) et le descend en (a) ; sur TD3 elle ne change rien en (b).

en ne changeant que `gradient_steps`.

Le résultat (tableau 3, figure 2) est monotone *pour chaque graine* dans les deux sens : le biais est multiplié par 25, la performance chute, et les trois exécutions à UTD=4 divergent. Sur les neuf exécutions, les corrélations de Spearman sont: $\rho(\text{UTD}, \text{biais}) = +0{,}94$ et $\rho(\text{biais}, \text{retour}) = -0{,}93$ ($p = 2 \cdot 10^{-4}$). Ces valeurs mettent en évidence un seuil de tolérance. L'agent parvient encore à atterrir correctement malgré un biais de 72 (ce qui lui donne un score positif de +58), mais sa politique d'action s'effondre totalement lorsque ce biais atteint 160 (entraînant un score négatif de −41).

Table 3: Expérience 2. DDPG sans LN, 100 000 pas, médiane des graines sur les trois derniers points. UTD=1 est l'expérience 1 tronquée au même budget.

UTD	graines	$\overline{Q-G}$	sd$(Q-G)$	Retour	Perte du critique
1	4	71,7	35,3	+57,9	33
2	2	159,9	70,5	−41,0	109
4	3	1824	686	−135,8	12 176

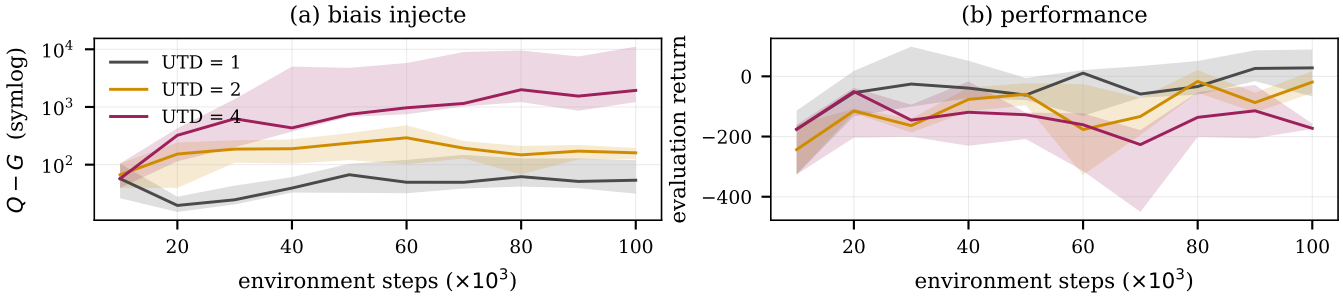


Figure 2: Expérience 2. DDPG sans LN, seul l'UTD varie. Médiane sur les graines, bande min–max.

Ce qu'on en conclut. Le biais cause bien une perte de performance : c'est notre premier argument causal, puisque rien d'autre n'a changé. Mais il ne la cause pas proportionnellement à son amplitude, et l'expérience situe un seuil : un biais de 72 laisse l'agent atterrir, un biais de 160 détruit sa politique. En revanche nous ne pouvons pas départager le niveau du biais de son inégalité, car les deux croissent ensemble ici : l'écart-type passe de 35 à 70 puis à 686 pendant que la moyenne passe de 72 à 160 puis à 1824, soit un peu plus vite. La prédiction de la section 2 reste donc une prédiction.

6 Portée et limites des corrélations observées

Le piège de la corrélation interne. Il ne faut pas calculer la corrélation entre le biais et la performance à l'intérieur d'une même exécution (au fil de l'entraînement). Si on le fait, on trouve une forte anticorrélation ($\rho = -0,60$). Cependant, ce calcul n'apporte aucune information réelle. En effet, le biais se calcule par la soustraction $Q - G$. Or, la valeur G (le retour actualisé) et le score de performance (le retour) sont mesurés sur les mêmes épisodes de test.Par conséquent, chercher la corrélation entre le biais $(Q - G)$ et le retour revient grossièrement à comparer mathématiquement $-G$ à G. L'anticorrélation que l'on observe est donc inévitable, et elle masque le comportement du réseau Q (qui, seul, va de −0,25 à +0,65 selon la configuration, donc sans relation stable). Pour analyser correctement l'impact du biais,il faut utiliser la corrélation inter-exécutions, c'est-à-dire comparer les entraînements entre eux. C'est là que la véritable dynamique se révèle : chez DDPG, qui souffre de surestimation (biais positif), la corrélation est de −0,93. Cela signifie que plus le biais est élevé par rapport aux autres exécutions, plus la performance s'effondre. À l'inverse, chez TD3, qui souffre de sous-estimation (biais négatif), la corrélation s'élève à +0,96. Dans ce cas, plus le biais remonte vers zéro pour devenir moins négatif, plus la performance augmente. Ces observations mènent la conclusion que peu importe que l'algorithme ait tendance à être trop optimiste ou trop pessimiste, rapprocher son estimation $\overline{Q}$ du retour effectivement obtenu $\overline{G}$ améliore le retour de l'agent.

Deux comportements opposés. Les deux familles échouent différemment. DDPG tente des manœuvres surévaluées et s'écrase vite : ses épisodes durent 353 pas en moyenne (241 avec LN). TD3 fait l'inverse et *stationne* : ses épisodes

durent 646 pas, et 40 % de ses pas de mesure proviennent d'épisodes tronqués, contre 9 % pour DDPG (figure 1c) ; dans la famille TD3, la performance est presque entièrement expliquée par ce comportement ($\rho(\text{troncature}, \text{retour}) = -0{,}93$). On pourrait penser qu'on peut attribuer ce surplace au pessimisme du critique mais l'expérience 4 montrera que c'est faux.

Ce qu'on en conclut. Nous écartons la corrélation qui est calculée à l'intérieur d'une exécution et est imposée par la construction de G. Ce qui reste montre que l'erreur d'estimation se manifeste dans les deux directions (surestimation ou sous-estimation). Dans chaque cas, rapprocher l'estimation Q de la réalité permet d'améliorer les performances. L'énoncé le plus rigoureux est donc qu'*à architecture fixée*, une estimation plus juste va avec un meilleur agent, et non qu'un biais positif élevé est systématiquement le seul problème. L'expérience 4 montrera que la relation cesse de tenir dès qu'on change un composant.

7 Expérience 3 :L'effet de la LN s'explique-t-il par la vitesse d'apprentissage ?

Il reste à expliquer pourquoi la LN nuit à DDPG. Une piste peut être dans la perte du critique. La LN la fait plus que doubler sur DDPG, de 26,1 à 65,5 (tableau 4), alors qu'elle ne la change pas sur TD3 (17,4 contre 16,7).. En normalisant les activations, la LN améliore le conditionnement et laisse le critique se déplacer plus vite à taux d'apprentissage fixé. Si c'est le mécanisme, baisser ce taux devrait annuler la dégradation. Nous avons donc relancé DDPG avec et sans LN à $3 \cdot 10^{-4}$, tout le reste inchangé et sur 5 graines comme précédamment.

Table 4: Expérience 3. Effet du taux d'apprentissage sur DDPG, valeurs finales, IQM sur 4 à 5 graines.

	Retour	$\overline{Q-G}$	sd$(Q-G)$	Perte du critique	Troncature
DDPG, 10^{-3}	158,0	47,6	35,0	26,1	0,03
DDPG + LN, 10^{-3}	−40,3	130,5	63,2	65,5	0,02
DDPG, $3 \cdot 10^{-4}$	14,1	55,3	33,8	15,8	0,26
DDPG + LN, $3 \cdot 10^{-4}$	−17,3	98,8	64,5	41,7	0,08

La prédiction n'est vérifiée qu'à moitié (tableau 4, figure 3). Ce qui la soutient : l'excès de biais dû à la LN se réduit d'un facteur 2,7 à un facteur 1,8, et la pénalité de performance tombe de −162 à −52, sous le seuil de détection (4 graines sur 5, test des signes $p = 0{,}19$; Mann–Whitney $p = 0{,}15$). Ce qui la réfute est que l'effet ne disparaît pas, puisqu'à $3 \cdot 10^{-4}$ la LN augmente encore le biais sur les **cinq** graines (+50,6, Mann–Whitney $p = 0{,}008$, la plus petite valeur de toute l'étude), ainsi que son écart-type et la perte du critique.

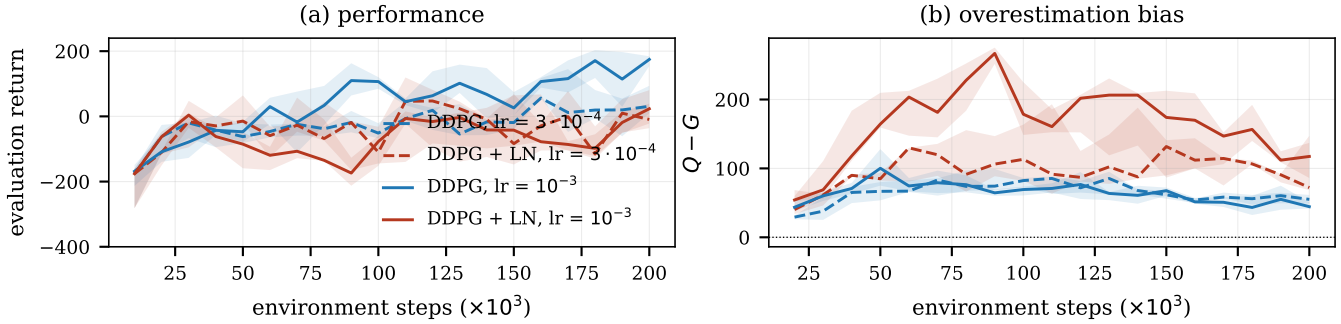


Figure 3: Expérience 3. DDPG avec et sans LN, aux deux taux d'apprentissage. Sans LN (bleu), les deux courbes de biais se superposent : le pas d'apprentissage ne le change pas. Avec LN (rouge), le biais reste au-dessus aux deux taux.

On peut ici faire deux remarques. D'abord, baisser le pas n'est pas un levier neutre puisque DDPG sans LN chute de 158 à 14 et deux de ses graines basculent dans le régime de surplace de la section 6 (troncature 0,80 et 0,73) et la comparaison oppose donc deux systèmes dégradés. Ensuite, diviser le pas par trois réduit la perte du critique de 26 à 16 *sans* réduire le biais (47,6 → 55,3, $p = 0{,}45$), alors que doubler le nombre de mises à jour par transition le multipliait par deux.

Ce qu'on en conclut. L'effet de la Layer Normalization ne se résume pas à une simple accélération de l'optimisation. Si la réduction du taux d'apprentissage atténue son impact, elle ne l'annule pas et la LN continue d'aggraver le biais sur toutes les graines, même à vitesse réduite. Cette expérience permet au passage d'isoler le véritable moteur du biais. Puisque la modification du taux d'apprentissage altère la vitesse d'optimisation sans changer le niveau du biais, ce dernier est bien causé par la réutilisation des données, et non par la vitesse d'apprentissage.

8 Expérience 4 : lequel des trois correctifs fait quoi ?

Pourquoi TD3 est-il pessimiste, et pourquoi sous-performe-t-il ? Nous retirons les trois correctifs un à un, 5 graines chacun, tout le reste identique à l'expérience 1 (tableau 5, figure 4). Les deux questions n'ont pas la même réponse.

Le double critique possède le biais, et lui seul. Le retirer fait passer le biais de −6,4 à +43,3 : le signe s'inverse sur les **cinq** graines (Mann–Whitney $p = 0{,}004$, la plus petite valeur de l'étude) et l'écart-type triple.Retirer le retard ne change *rien* au biais : il passe de −6,4 à −7,2, un décalage de −0,4 concordant sur 3 graines sur 5 seulement, donc indiscernable du bruit. Retirer le lissage ramène le biais à +0,46, soit un décalage de +5,6 concordant sur 4 graines sur 5, sans atteindre notre seuil ($p = 0{,}075$).

L'ordre de grandeur est celui que prédit l'équation (1) : prendre le minimum plutôt que la moyenne des deux critiques

coûte 0,65 unité de valeur, et $1/(1-\gamma) = 50$ fois cela donne un décalage attendu d'environ 33, pour un décalage observé de 50.L'accord n'est qu'en ordre de grandeur : retirer un critique ne revient pas exactement à remplacer le minimum par la moyenne, puisque le critique restant a sa propre erreur.

Mais ce sont les deux autres qui possèdent la performance. Retirer le retard porte le retour de 44,6 à 193,6 (+100, Mann–Whitney $p = 0{,}048$), retirer le lissage à 133,6 (+91, **5** graines sur 5). Retirer le double critique ne change rien (+4, 3 graines sur 5).C'est le seuil de la section 5 vu de l'autre côté : +50 de biais injecté ici ne coûte rien, là où +160 y détruisait la politique. Nos deux interventions, par deux leviers différents, situent donc la rupture entre quelques dizaines et quelques centaines d'unités de retour. Les trois ablations réduisent par ailleurs la troncature, de 0,51 à 0,11–0,16. Aucune de ces ablations ne cause le surplace à elle seule. Une réserve sur la deuxième pourrait être que dans Stable-Baselines3 les mises à jour douces des réseaux cibles sont à l'intérieur du bloc retardé, de sorte que passer à $d = 1$ accélère aussi les cibles. Nous avons donc levé deux choses à la fois.

Table 5: Expérience 4. Valeurs finales, IQM sur 5 graines, intervalle bootstrap à 95 % pour les deux premières colonnes. La dernière colonne compte les graines dont le biais final est positif.

	Retour	$\overline{Q-G}$	sd$(Q-G)$	Troncature	biais > 0
TD3 complet	44,6 [−28, 145]	−6,44 [−8,8, −5,0]	13,4	0,51	0/5
sans double critique	60,2 [−9, 122]	+43,3 [36, 56]	37,4	0,11	**5/5**
sans retard de l'acteur	**193,6** [42, 222]	−7,17 [−10,0, −4,1]	16,3	0,11	0/5
sans lissage de cible	133,6 [89, 224]	+0,46 [−7,8, +3,7]	17,9	0,16	3/5

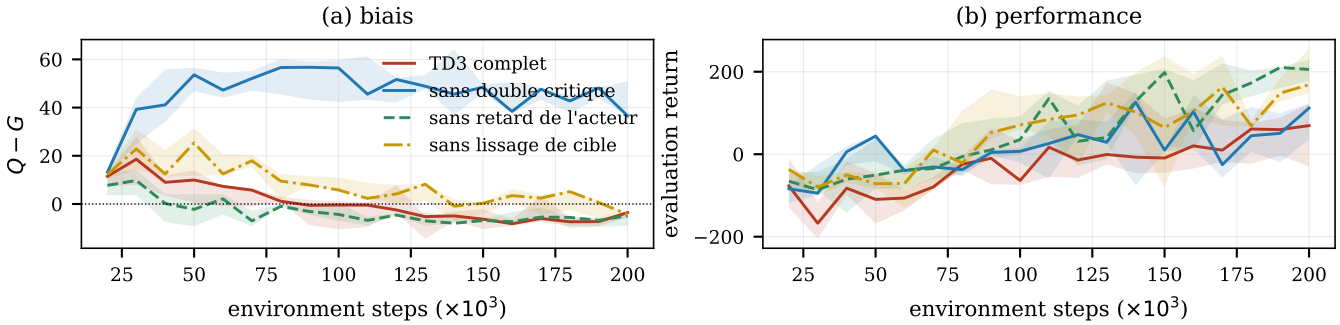


Figure 4: Expérience 4. Médiane sur les graines, bande interquartile. Seule l'ablation du double critique (bleu) laisse le biais franchement positif en (a), alors que ce sont les deux autres qui montent en (b).

Ce qu'on en conclut. Le composant qui contrôle le biais ne contrôle pas la performance, et réciproquement. Si TD3 sous-performe ici, ce n'est donc pas à cause de son biais, puisque supprimer ce biais ne l'améliore pas. Cela réfute aussi l'interprétation de la section 6 puisque retirer le retard laisse le biais à −7,2 et fait pourtant disparaître le surplace, qui n'est donc pas causé par le pessimisme. La cause pourrait donc être qu'un acteur qu'un mis à jour une fois sur deux apprend simplement trop lentement pour un budget de 200 000 pas (voir les limites de notre étude).

9 Conclusion et limites

Question 1. La layer normalization n'a pas d'effet uniforme sur le biais. Elle le multiplie par 2,7 dans DDPG, où elle dégrade fortement la performance, et le laisse inchangé dans TD3, où elle l'améliore pourtant sur toutes les graine. Son bénéfice y est d'une autre nature. Baisser le taux d'apprentissage atténue cet excès sans le supprimer, donc il n'est qu'en partie un effet de vitesse d'optimisation.

Question 2. Le biais dégrade bien la performance, mais pas proportionnellement à son amplitude : la manipulation de l'UTD situe la rupture entre un biais de 72, inoffensif, et un biais de 160, destructeur. Nos deux interventions se recoupent sur ce seuil : l'ablation du double critique injecte +50 de biais et ne coûte rien (+4 de retour, 3 graines sur 5), ce qui place le seuil au-delà de 50 et en deçà de 160. En revanche nos manipulations déplacent toujours ensemble le niveau du biais et son écart-type, si bien que nous ne pouvons pas attribuer le dommage à l'un plutôt qu'à l'autre : la prédiction de la section 2, selon laquelle seule l'inégalité du biais devrait compter, n'est pas testée par cette étude. Le lien n'est pas universel non plus, puisque l'ablation de TD3 exhibe un cas où déplacer le biais ne change pas la politique, parce que ce qui limitait l'agent était ailleurs.

Limites. Bien que nos résultats aillent tous dans la même direction, notre budget de calcul (quatre ou cinq graines) borne la significativité atteignable : le test des signes ne descend pas sous 0,031, le Mann–Whitney pas sous 0,004. Nos effets sont unanimes en signe, mais leur signification est limitée par ce budget plutôt que par leur taille. De plus, la portée de notre étude reste restreinte par ses paramètres : nous n'avons testé qu'un unique environnement, une seule valeur d'actualisation γ, et des entraînements arrêtés à 200000 pas, peut être avant que les algorithmes n'atteignent leur plateau de performance. Avec des ressources de calcul supérieures, notre première démarche serait d'ailleurs de prolonger ces entraînements pour observer le comportement à long terme des agents.Par ailleurs, notre méthode d'ablation retire les composants un par un, ce qui empêche d'observer leurs éventuelles interactions.

Références. Agarwal et al., NeurIPS 2021. -Ba et al., arXiv 2016. - Chen et al., ICLR 2021 (REDQ). - Colas, Sigaud et Oudeyer, arXiv 2019. - Fujimoto et al., ICML 2018 (TD3). - Lillicrap et al., ICLR 2016 (DDPG). - Nauman et al., ICML 2024. - Pardo et al., ICML 2018. - Raffin et al., JMLR 2021 (Stable-Baselines3) et *RL Baselines3 Zoo*.